

Documentación del Framework Tecnológico

Proyecto Consultorio Jurídico ICESI

Proyecto de Grado - Metodología SCRUM

Sprint 2 - 2025

1. Introducción

El presente documento describe el framework tecnológico adoptado para el desarrollo del proyecto Consultorio Jurídico y detalla como cada uno de sus componentes se utiliza dentro del sistema. El framework seleccionado es Django, un framework de alto nivel para Python que sigue el principio de batteries included, cuyo uso fue definido como obligatorio desde el inicio del proyecto por los requerimientos del equipo académico.

El objetivo de este documento es dejar por escrito como Django esta siendo utilizado en el proyecto: que componentes activos tiene, como se estructura el código en torno a ellos, y como se aplican al contexto específico del Consultorio Jurídico con sus modelos de datos, roles de usuario y funcionalidades de gestión de citas.

2. Por que Django es el framework correcto para este proyecto

Django fue adoptado porque ofrece de forma nativa exactamente lo que el Consultorio Jurídico necesita: un ORM para gestionar relaciones complejas entre citas, casos y usuarios; un sistema de autenticación listo para usar; un mecanismo de roles extensible; templates HTML integrados; y un panel de administración configurable. Todo esto en un solo framework, sin dependencias adicionales.

La alternativa de separar frontend y backend en dos proyectos independientes (por ejemplo, Django como API REST y React como frontend) fue descartada porque duplica la complejidad de desarrollo y despliegue sin aportar ventajas reales para el alcance del sistema. Un sistema de gestión de citas jurídicas con cinco roles, formularios y un calendario no requiere la reactividad de un SPA.

Del mismo modo, para la actualización del calendario en tiempo real (HU12), se adoptó una estrategia de polling con fetch y setInterval en lugar de WebSockets. Esto significa que el frontend consulta un endpoint Django cada 8 segundos para verificar cambios en el estado de las citas. Esta solución no requiere instalar Django Channels ni Redis, funciona idénticamente en local y en despliegue, y es suficiente dado que una cita jurídica no cambia de estado cada segundo.

3. Componentes de Django en uso en el proyecto

La siguiente tabla describe cada componente de Django activo en el proyecto, su función específica dentro del Consultorio Jurídico y el archivo o módulo donde se encuentra implementado:

Componente de Django	Función dentro del proyecto	Donde se aplica en el código
ORM (Object-Relational Mapper)	Mapea los modelos Python a tablas de base de datos y gestiona las consultas sin SQL manual.	citas/models.py: modelos Cita, Caso, HorarioDisponible, RegistroAsistencia. usuarios/models.py: modelo Usuario.
Sistema de migraciones	Genera y aplica cambios al esquema de la base de datos de forma incremental y versionada.	citas/migrations/, usuarios/migrations/, consentimiento/migrations/
Autenticación nativa	Gestiona login, logout, sesiones y protección de vistas mediante decoradores.	usuarios/views.py: login_view, logout_view. Decorador @login_required en todas las vistas protegidas de citas/views.py.
Modelo de usuario personalizado	Extiende AbstractUser para eliminar el campo username y agregar documento, correo, nombre_completo, teléfono, dirección y rol.	usuarios/models.py: clase Usuario con USERNAME_FIELD = 'documento' y campo rol con RolChoices.
Sistema de roles	Define los roles del sistema (beneficiario, estudiante, profesor, administrador, secretaria) como campo texto del modelo Usuario, usado para filtrar vistas y permisos.	usuarios/models.py: RolChoices. citas/views.py: condiciones por request.user.rol en gestionar_citas_view y marcar_asistencia_view.
Vistas basadas en funciones	Controlan la lógica de cada pantalla del sistema, reciben la petición HTTP, consultan el modelo y retornan un template renderizado.	citas/views.py: home_view, gestionar_citas_view, gestionar_casos_view, agendar_cita_view, cancelar_cita_view, posponer_cita_view, confirmar_cita_view, marcar_asistencia_view.
Sistema de templates	Renderiza el HTML en el servidor con datos del contexto de la vista. Permite herencia de templates para no repetir estructura.	templates/base_dashboard.html como plantilla base. citas/templates/citas/gestionar_citas.html y gestionar_casos.html como templates específicos.
Sistema de URLs	Mapea cada ruta HTTP a su vista correspondiente mediante patrones de URL nombrados.	consultorio/urls.py: enrutador principal. citas/urls.py: rutas de citas y casos. usuarios/urls.py: rutas de login y registro.
Formularios Django	Validan y procesan los datos enviados por el usuario desde los templates HTML.	usuarios/forms.py: LoginForm, RegistroPaso1Form, RegistroPaso2Form. citas/views.py:

Componente de Django	Función dentro del proyecto	Donde se aplica en el código
		procesamiento de POST con validaciones manuales.
Mensajes flash (messages)	Muestra notificaciones de éxito o error al usuario tras realizar una acción (agendar, cancelar, confirmar).	Usado con <code>messages.success()</code> y <code>messages.error()</code> en todas las vistas de acción de citas/ <code>views.py</code> .
Panel de administración	Interfaz web para gestionar directamente los registros de la base de datos sin código adicional.	<code>citas/admin.py</code> , <code>usuarios/admin.py</code> : registro de modelos para visualización y edición desde <code>/admin/</code> .
Endpoint JSON para polling	Vista Django que devuelve el estado actual de las citas en formato JSON, consultada cada 8 segundos por el frontend para actualizar el calendario sin recargar la página.	Por implementar en <code>citas/views.py</code> como vista adicional. Retorna <code>JsonResponse</code> filtrado según el rol del usuario autenticado.

4. Estructura de modelos de datos

El modelo de datos del Consultorio Jurídico está definido en Django ORM. Cada clase Python representa una tabla en la base de datos. Las relaciones entre modelos se gestionan con `ForeignKey` y `OneToOneField`, y Django genera automáticamente las migraciones cuando se realizan cambios en los modelos.

Modelo	Archivo	Campos principales
Usuario	<code>usuarios/models.py</code>	documento (PK), nombre_completo, correo, telefono, direccion, rol (RolChoices), acepta_tratamiento_datos
Caso	<code>citas/models.py</code>	codigo, beneficiario (FK Usuario), sala_juridica, descripcion, estado, estudiante_asignado (FK Usuario, rol=estudiante), fecha_creacion
HorarioDisponible	<code>citas/models.py</code>	fecha, hora_inicio, hora_fin, disponible (bool). unique_together: fecha + hora_inicio
Cita	<code>citas/models.py</code>	beneficiario (FK Usuario), caso (FK Caso), horario (FK HorarioDisponible), tipo_atencion, estado (TRANSICIONES_VALIDAS), motivo, fecha_creacion, fecha_confirmacion, fecha_cancelacion
RegistroAsistencia	<code>citas/models.py</code>	cita (OneToOne), asistio (bool), fecha_registro, registrado_por (FK Usuario), observaciones

4.1 Lógica de negocio encapsulada en el modelo Cita

El modelo Cita implementa un sistema de maquina de estados mediante el diccionario TRANSICIONES_VALIDAS, que define que cambios de estado son legales:

- pendiente puede pasar a: confirmada, cancelada
- confirmada puede pasar a: cumplida, no_asistio, cancelada
- cumplida, no_asistio y cancelada son estados terminales sin transiciones posibles

El metodo cambiar_estado() valida esta logica antes de guardar y gestiona efectos secundarios como registrar la fecha de confirmacion, registrar la fecha de cancelacion y liberar el HorarioDisponible cuando una cita se cancela. Esta logica esta centralizada en el modelo, por lo que todas las vistas la heredan sin repeticion de codigo.

5. Roles de usuario y control de acceso

El control de acceso en Django se implementa mediante el campo rol del modelo Usuario y el decorador @login_required en las vistas. La siguiente tabla resume los cinco roles del sistema, las vistas a las que tienen acceso, y el filtro de datos que se les aplica en cada consulta:

Rol	Vistas accesibles	Filtro de datos aplicado	Permisos especiales
beneficiario	home_view, gestionar_citas_view, gestionar_casos_view, agendar_cita_view	beneficiario=request.user en Cita y Caso	Puede confirmar, cancelar y posponer sus propias citas
estudiante	Vista propia pendiente de implementacion en Sprint 2	caso__estudiante_asignado=request.user en Cita	Solo lectura sobre las citas de sus casos asignados
profesor	Vista propia pendiente de implementacion en Sprint 2	Por definir segun relacion con salas juridicas	Supervision de casos y citas de su area
administrador	gestionar_citas_view con es_admin=True	Sin filtro: accede a todas las citas del sistema	Puede marcar asistencia y no asistencia (marcar_asistencia_view)
secretaria	gestionar_citas_view con es_admin=True	Sin filtro: accede a todas las citas del sistema	Puede marcar asistencia y no asistencia (marcar_asistencia_view)

El rol se verifica directamente en la vista mediante request.user.rol. Por ejemplo, en gestionar_citas_view, la variable es_admin agrupa a administrador y secretaria para mostrarles todas las citas, mientras que cualquier otro rol solo ve las propias. Este patron se replica en la vista de marcar_asistencia_view, donde unicamente administrador y secretaria pueden registrar asistencia o inasistencia.

6. Estrategia de frontend con Django Templates

El frontend del Consultorio Juridico se construye con el sistema de templates nativo de Django. El flujo de renderizado es el siguiente: el usuario hace una peticion HTTP, la vista Django consulta el modelo, prepara el contexto y retorna el template HTML ya renderizado con los datos al navegador. No hay una capa adicional de JavaScript reactivo para la mayoria de las pantallas.

6.1 Herencia de templates

Todos los templates del sistema heredan de templates/base_dashboard.html mediante la directiva extends. Este archivo base contiene la estructura comun: barra de navegacion, estilos globales y bloques reutilizables (title, extra_styles, content). Cada template especifico solo define el contenido de su bloque content, evitando duplicacion de HTML.

6.2 Actualizacion en tiempo real con polling

Para la HU12 (actualizacion automatica del calendario en tiempo real), el sistema utiliza un endpoint Django que devuelve el estado de las citas en JSON. El template correspondiente a cada rol incluye un bloque JavaScript que ejecuta fetch() a ese endpoint cada 8 segundos mediante setInterval. Si la respuesta contiene cambios respecto a lo que el DOM muestra, el script actualiza unicamente los elementos afectados sin recargar la pagina completa.

Este endpoint es una vista Django normal decorada con @login_required que retorna JsonResponse. El filtro de datos dentro de esta vista respeta el rol del usuario autenticado: beneficiarios ven solo sus citas, estudiantes ven las de sus casos asignados, y administradores y secretarias ven todas. Esto garantiza que el mecanismo de tiempo real no exponga datos de otros usuarios.

6.3 Formularios y acciones POST

Las acciones del sistema (agendar cita, cancelar, confirmar, marcar asistencia) se realizan mediante formularios HTML con method POST. Django valida el token CSRF automaticamente en cada peticion POST, protegiendo el sistema contra ataques de falsificacion de solicitudes entre sitios. Tras procesar la accion, la vista redirige al usuario con un mensaje flash que se muestra en el siguiente template cargado.

7. Configuracion y dependencias del proyecto

El archivo requirements.txt del proyecto declara una unica dependencia: django==6.0.3. Esta es la version activa del framework en el proyecto. No se requieren librerias adicionales para las funcionalidades implementadas ni para las previstas en el Sprint 2.

La configuracion principal del proyecto esta en consultorio/settings.py, que define:

- AUTH_USER_MODEL = 'usuarios.Usuario': indica a Django que use el modelo de usuario personalizado en lugar del modelo por defecto.
- INSTALLED_APPS: registra todas las aplicaciones del proyecto (citas, usuarios, consentimiento, reportes).

- DATABASES: configurado con SQLite para desarrollo local. Para el despliegue de prueba, esta configuracion puede cambiarse a PostgreSQL sin modificar el codigo de los modelos.

El enrutamiento principal esta en `consultorio/urls.py`, que incluye los `URLconf` de las aplicaciones citas y usuarios. El archivo `consultorio/asgi.py` esta presente pero el proyecto corre actualmente con WSGI (`consultorio/wsgi.py`), lo que es suficiente para el alcance del sistema sin necesidad de un servidor asincrono.

8. Consideraciones de despliegue

Para el despliegue de prueba, el proyecto Django se sirve con Gunicorn como servidor WSGI. La configuracion de base de datos se cambia a PostgreSQL en el `settings.py` del entorno de produccion mediante variables de entorno, sin modificar el codigo de los modelos gracias a que el ORM de Django abstrae completamente el motor de base de datos.

Plataformas como Railway o Render soportan este esquema directamente: detectan el proyecto Django, instalan las dependencias de `requirements.txt`, ejecutan las migraciones y levantan el servidor Gunicorn. No se requiere configuracion de servidores de mensajeria como Redis ni de servidores asincronos como Daphne, dado que el sistema no usa WebSockets.

9. Conclusiones

Django cubre todos los requerimientos del Consultorio Juridico con sus componentes nativos. El ORM gestiona el modelo de datos, el sistema de autentificacion maneja los cinco roles, el sistema de templates construye la interfaz, y el mecanismo de polling con `fetch` provee las actualizaciones en tiempo real sin dependencias adicionales. La arquitectura es de un solo proyecto, un solo repositorio y un solo despliegue, lo que reduce la friccion operativa y permite que el equipo se concentre en implementar las historias de usuario del backlog.

Referencias

- Django Project. (2024). Django Documentation. <https://docs.djangoproject.com/>
- Django Project. (2024). The Django ORM. <https://docs.djangoproject.com/en/6.0/topics/db/models/>
- Django Project. (2024). Authentication in Django. <https://docs.djangoproject.com/en/6.0/topics/auth/>
- Django Project. (2024). Django Templates. <https://docs.djangoproject.com/en/6.0/topics/templates/>
- MDN Web Docs. (2024). Fetch API y uso de `setInterval`. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- Railway. (2024). Despliegue de aplicaciones Django. <https://railway.app/>